

NOURISH NETWORK

Basic Overview & Complete Technical Deep-Dive Manual

PART 1: BASIC OVERVIEW OF NOURISH NETWORK

1. What is Nourish Network?

Nourish Network is a real-time digital surplus food allocation ecosystem. It connects food vendors (restaurants, caterers, hotels) directly with verified NGOs, shelters, and community organizations to redistribute safe, edible surplus food before it spoils.

2. The 3-Step Simple Workflow

- * **Step 1 (List Surplus):** Food vendors log into their portal and list available surplus meals, specifying quantity, pickup window, and FSSAI safety license code.
- * **Step 2 (Instant Alert):** The backend engine instantly dispatches real-time email notifications to verified NGOs in the surrounding area.
- * **Step 3 (Claim & Pickup):** Verified NGOs claim the meal listing on their dashboard, enabling fast coordination and pickup.

3. Basic Technical Concepts Explained Simply (For Teammates)

- * **Frontend (The Screen & Buttons):** Everything the user sees and clicks on their screen (HTML layout, CSS styling, JavaScript clicks). Files: index.html, dashboard.html, script.js, styles.css.
- * **Backend (The Hidden Engine):** The server running behind the scenes (server.js using Node.js & Express). It verifies passwords, saves food listings, and sends email alerts.
- * **Database (The Digital Filing Cabinet):** The place where all data (user accounts, food listings, claimed orders) is permanently stored so it is not lost when the browser closes.
- * **API (The Restaurant Waiter):** An API (Application Programming Interface) carries requests from the frontend to the backend and brings back the response. Analogy: You (Frontend) order food from the Waiter (API), who takes it to the Kitchen (Backend) and brings back your meal.
- * **REST API (Standardized Menu):** A REST API follows standard web methods: POST /api/register (create account), POST /api/login (sign in), GET /api/listings (fetch meals), POST /api/orders (claim meal).
- * **API Key (VIP Access Pass):** A secret password key that allows your software to talk to third-party services securely (e.g., Google Firebase API keys).

PART 2: TECHNICAL DEEP-DIVE & SYSTEM ARCHITECTURE

Q1: Why are there TWO databases (PostgreSQL + SQLite) in your code?

- * Local SQLite (database.sqlite): Perfect for local offline testing on your laptop without internet or cloud credentials. Runs in WAL (Write-Ahead Logging) mode for fast concurrent operations.
- * Cloud PostgreSQL (Supabase): When deployed on live cloud hosts (Render.com), free servers restart often and wipe local laptop files. PostgreSQL on Supabase keeps all data permanently safe in the cloud.
- * In code (database.js): The server checks process.env.DATABASE_URL. If present, it connects to Supabase PostgreSQL; otherwise, it falls back to local SQLite.

Q2: What is a Salt Round? (SALT_ROUNDS = 10)

- * Plaintext passwords like "myPass123" are NEVER stored. They are scrambled (hashed) into gibberish using bcryptjs.
- * A "Salt" is a random string added to the password before scrambling so two users with "123456" get completely different hashes.
- * "10 Salt Rounds" means the algorithm scrambles the password $2^{10} = 1,024$ times in a loop! This prevents hacker supercomputers from guessing passwords, while real users log in in 0.05 seconds.

Q3: Is it Verified Vendor or Verified NGO?

- * BOTH are verified, but for different safety purposes!

* Verified Vendors (Food Safety): Vendors must enter a valid 14-digit FSSAI Code. Your system decodes and verifies their license to ensure food safety.

* Verified NGOs (Targeted Alerts): When vendors post surplus food, server.js sends real-time email alerts ONLY to verified NGOs (isVerified = 1) who activated their account. This prevents spammers from taking free community meals.

Q4: What is the Dual-Engine Email Dispatcher?

* Engine 1 (Google Apps Script HTTPS Bridge - Primary): Cloud hosts like Render block SMTP ports (587/465). mailer.js calls a Google Apps Script HTTPS URL on standard Web Port 443 to send emails smoothly.

* Engine 2 (Nodemailer SMTP - Fallback): When testing locally on a laptop, mailer.js uses standard Gmail SMTP.

* Combined Result: 100% reliable email delivery on both cloud and local environments!

Q5: Step-by-Step Architecture Execution Flow

1. User Action: Vendor posts surplus food or NGO claims a meal.
2. Express REST API: Frontend sends POST request payload to /api/listings or /api/orders.
3. Database Sync: Server inserts/updates row in food_listings or orders tables.
4. Email Dispatch: Asynchronous background trigger sends notifications via Google Apps Script HTTPS Bridge.
5. Role Steering: auth/handler.js checks account classification and routes the user to seller-dashboard.html or buyer-dashboard.html.